Chapter 4: Implementing View Classes

This chapter examines the different types of view classes available in the Microsoft Foundation Class (MFC) Library. The chapter begins with a general discussion of how views fit into the document/view architecture, and continues with sections on some of the views.

The **CHTMLView** class, which provides Internet browsing functionality, is not covered in this chapter. For information about **CHTMLView**, see Implementing HTML Views in Chapter 9: Building Internet Applications.

Objectives

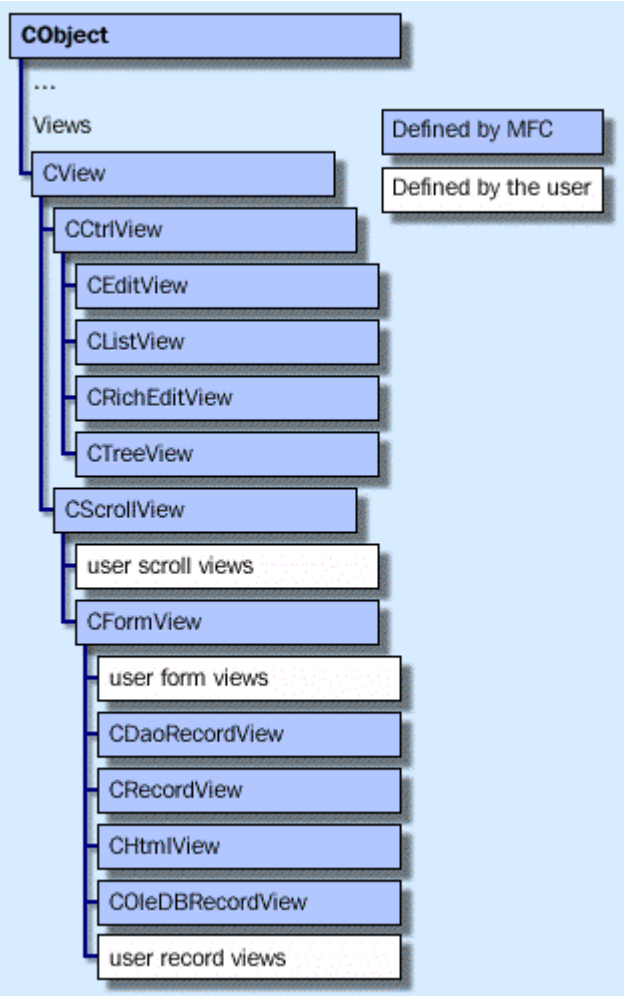
After completing this chapter, you will be able to:

- Describe the purpose of documents, views, templates, and frames within the document/view architecture, and explain how they interact.
- Describe the various types of view classes in MFC.
- Implement applications that use the following view classes:
  - **CView**
  - **CScrollView**
  - **CListView**
  - **CSplitter**
  - **CTreeView**
  - **CEditView**
  - **CRichEditView**
- Create single-document interface (SDI) and multiple-document interface (MDI) applications with multiple views.
- Use two interrelated views in an application.

84 Introduction to Views

The view class, **CView**, physically represents the client area of the application. Logically, **CView** represents a viewport of the information contained in the document class. It also allows user input through the mouse or keyboard.

MFC supports a wide variety of views. To see an illustration of the hierarchy of MFC view classes, click this icon.
(CD-ROM displays the following illustration.)



By deriving your application's view from one of these classes, the view inherits support for interaction with the application framework. This section provides information on the **CView** class and how it fits into document/view architecture. It also introduces the relationship between the view and the document.

Notes

The **CView** class simplifies the implementation of printing and print preview for your documents. For more information about implementing printing features, see Chapter 10: Printing and Print Preview.

This section includes the following topics:

- The CView Class
- Managing Document/View Interaction

Notes

85 The CView Class

The **CView** class provides the basic functionality for user-defined view classes.

A view is attached to a document and acts as an intermediary between the document and the user. The view renders an image of the document on the screen or printer and interprets user input as operations on the document.

A view is a child of a frame window. More than one view can share a frame window, as in the case of a splitter window. When the user opens a new window or splits an existing one, the framework constructs a new view and attaches it to the document.

Relationship Between a View and a Document

A view can be attached to only one document, but a document can have multiple views attached to it at once, as when the contents of the document are displayed in a splitter window or in multiple child windows in an MDI application.

A view displays a document's data. The document provides the view with the necessary details about its data. You can have the view access the document's data members directly, or you can provide member functions in the document class for the view class to call.

Coordinating the Views of a Document

To manage the complex process of creating documents with their associated views and frame windows, the framework uses two document template classes:

- **CSingleDocTemplate**

  You use **CSingleDocTemplate** with single-document interface (SDI) applications to create and store one document of one type.

- **CMultiDocTemplate**

  You use **CMultipleDocTemplate** with multiple-document interface (MDI) applications to create and store multiple documents of one type.

When a document's data changes, the view responsible for the changes typically calls the **CDocument::UpdateAllViews** function for the document, which notifies all the other views by calling the **OnUpdate** member function for each view. The default implementation of **OnUpdate** invalidates the view's entire client area. You can override this function to invalidate only those parts of the client area that you want to redraw.

Notes

86 Managing Document/View Interaction

To manage the interaction between a document and its view, you must complete the following steps.

1. Establish communication between the document and view.
2. Display data in the view.
3. Update the view when data in the document changes.

Establishing Communication

To establish communication between the current document and a view, create a pointer to the appropriate type of document and call the view's **GetDocument** member function, as shown in the following example code:

```
// Samples\CH04\Mdiapp
// Mdiview.cpp
void CMDIAppView::OnDraw(CDC* pDC)
{
    // Get pointer to current document
    CMDIAppDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    // Use document pointer as required
    // to access document data and functions

    // ... remainder of drawing code goes here
}
```

Notes

Displaying Data

To display data in a view, override the **OnDraw** function. To see sample code that overrides the **OnDraw** function, click this icon.

 (CD-ROM displays the following sample code.)

```
// Overriding the OnDraw function to display data in a view
// Samples\CH04\Mdiapp
// Mdiview.cpp

void CMDIAppView::OnDraw(CDC* pDC)
{
    // Get pointer to current document
    CMDIAppDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    // Calculate position to display text
    CRect r;
    GetClientRect(&r);
    int x = r.right / 2, y = r.bottom / 2;

    // Set text characteristics
    pDC->SetTextColor(pDoc->GetColor());
    pDC->SetTextAlign (TA_CENTER | TA_BASELINE);

    // Display text to the view
    pDC->TextOut (x, y, pDoc->GetPhrase());
}
```

Updating the View

To update the view(s) when data in the document changes, use either one of these procedures:

- **CWnd::Invalidate** (or **CDocument::UpdateAllViews**)

  Use in applications that use a single view.
- **CDocument::UpdateAllViews**

  Use in applications that use multiple views.

Notes

In applications that use views of different types, calling the **UpdateAllViews** function may not be the easiest solution for updating data. For information about a method that bypasses the **UpdateAllViews** function, see Coordinating Multiple Interrelated Views in this chapter.

 (CD-ROM displays the following tip.)

You can use the pHint and lHint parameters of **UpdateAllViews** to pass information to the views about the modifications made to the document. You can encode information by using lHint, and you can define a **CObject**-derived class to store information about the modifications and pass an object of that class by using pHint. Override the **CView::OnUpdate** member function in your **CView**-derived class to optimize the updating of the view's display, based on the information passed.

For more information about the **UpdateAllViews** function, search for "UpdateAllViews" in MSDN Library Visual Studio 6.0.

To see sample code that shows how an MDI application uses the **UpdateAllViews** function to ensure that all views associated with the current document are correctly repainted when the user changes the color of the text displayed in the view, click this icon.

 (CD-ROM displays the following sample code.)

```
// Repainting all views of a document when the user changes the text
// color in one of the views
// Samples\CH04\Mdiapp
// Mdiview.cpp

 void CMDIAppView::OnColors(UINT nID)
{
    CMDIAppDoc* pDoc = GetDocument();
    pDoc->SetColor(IDtoColorRef(nID));

    // If this were an SDI app then you could call
        // Invalidate();
    // Since it's an MDI app, you have to call
    pDoc->UpdateAllViews(NULL);
}
```

For more information about functions used to update views, search for "Updating Views" in MSDN Library Visual Studio 6.0.

Notes

87 Adding Multiple Views

It is possible to support more than one view of a single document. To help you implement multiple views, a document object keeps a list of its views, provides member functions for adding and removing views, and supplies the **UpdateAllViews** member function, which sends a message to the views when the user changes the data in a document.

This section explains how to create multiple views of the same document for SDI and MDI applications. In the sample applications for this section, multiple views share a single frame window. The views are constructed from different classes, each view providing a different way to display the same document. For example, one view might show the text in a document in a normal mode, while the other view shows the text in italic.

This section includes the following topics:

- Adding Multiple Views to an SDI Application
- Adding Multiple Views to an MDI Application
- Suppressing Automatic Creation of a Child Window
- Setting Titles for Child Frame Windows

Notes

88 Adding Multiple Views to an SDI Application

In an SDI application, you can have multiple views of the same document. The views are constructed from different classes, each view providing a different way to display the same document. Several steps are required to add a second view to an SDI application.

▶ **To create an SDI application with two views of the same document**

1. Build an SDI application the way you normally would. For an application with more than one view class, like the one discussed in these steps, place the command handlers in the **CMainFrame** class. Placing the command handlers in **CMainFrame** ensures that they will be available regardless of which view is active. If you place the command handlers in the view class, the handlers will not be available unless that view object is active.

    (CD-ROM displays the following tip.)

   In an application with only one view class, place command handlers in the view class. In an application with more than one view class, place command handlers in the **CMainFrame** (SDI) or **CChildFrame** (MDI) class. The functions **CFrameWnd::GetActiveView** and **CFrameWnd::GetActiveDocument** facilitate accessing the view of the document from your main frame or child frame.

2. Add a new view class derived from **CView**.
3. Add debug and nondebug versions of the **GetDocument** function.

   Use the **GetDocument** functions from your existing view class as a model.
4. Add a public constructor.

   ClassWizard provides the new view class with a protected constructor. Make sure the constructor differs from the existing constructor in a way other than scope.
5. Make the destructor provided by Class Wizard public.

   The protected destructor is not necessary, and the new constructor you have added is already public.
6. Include the document class header file in the source file of the new view class.
7. Add a handler for the **OnDraw** event of the new view class.
8. Include the header file of the new view class in Mainfrm.cpp.

Notes

9. Add two protected variables in the **CMainFrame** class.

   These variables will be pointers to the two view objects your application uses. Initialize these pointers to NULL (0) in the constructor of **CMainFrame**, as shown in the following example code:

```
class CMainFrame : public CFrameWnd
{
    ...
protected:
    CItalicsView * m_pItalicsView;
    CDefaultView * m_pDefaultView;
    ...
}
CMainFrame::CMainFrame()
{
    // To give this SDI app 2 views, the main frame stores
    // pointers to them both.
    m_pItalicsView = 0;
    m_pDefaultView = 0;
}
```

10. Add two menu selections to permit the application to switch between the two views.

    The **View** menu is a good place to insert the menu selections, but you can choose any of the drop-down menus.

11. Add command handlers to **CMainFrame** for the two new menu items.

Notes

To see sample code that shows how to add the command handler to switch to the new view, click this icon.

 (CD-ROM displays the following sample code.)

```
// To switch to the new view and hide the default view
// Samples\CH04\SDI2VIEWS
void CMainFrame::OnViewItalics()
{
    CDocument* pDoc = GetActiveDocument();

    // Only need to do this one time.
    if (0 == m_pItalicsView)
    {
        m_pDefaultView = (CDefaultView *)GetActiveView();
        // This object is destroyed in main frame's d'tor.
        m_pItalicsView = new CItalicsView(NULL);

        m_pItalicsView->Create(NULL, NULL, AFX_WS_DEFAULT_VIEW,
                rectDefault, this, AFX_IDW_PANE_FIRST + 1);
    }

    pDoc->AddView(m_pItalicsView);

    // Set the child i.d. of the italics view to AFX_IDW_PANE_FIRST,
    // so that CFrameWnd::RecalcLayout will allocate to this
    // "first pane" that portion of the frame window's client area
    // not allocated to control bars.  Set the child i.d. of the
    // default view to AFX_IDW_PANE_FIRST + 1
    m_pItalicsView->SetDlgCtrlID(AFX_IDW_PANE_FIRST);
    m_pDefaultView->SetDlgCtrlID(AFX_IDW_PANE_FIRST+1);

    // Show the italics view and hide the default view.
    m_pItalicsView->ShowWindow(SW_SHOW);
    m_pDefaultView->ShowWindow(SW_HIDE);

    SetActiveView(m_pItalicsView);

    // See comment in OnViewDefault.
    pDoc->RemoveView(m_pDefaultView);
    RecalcLayout();
}
```

Notes

To see sample code that shows how to add the command handler that restores the default view, click this icon.

 (CD-ROM displays the following sample code.)

```
// To switch to the default view and hide the second view (the italics
// view)
// Samples\CH04\SDI2VIEWS
void CMainFrame::OnViewDefault()
{
    CDocument* pDoc = GetActiveDocument();

    // The UI handler will prevent this function executing before
    // OnViewItalics, but just in case...
    ASSERT (m_pItalicsView != 0);
    ASSERT (m_pDefaultView != 0);

    pDoc->AddView(m_pDefaultView);

    m_pDefaultView->ShowWindow(SW_SHOW);
    m_pItalicsView->ShowWindow(SW_HIDE);

    m_pDefaultView->SetDlgCtrlID(AFX_IDW_PANE_FIRST);
    m_pItalicsView->SetDlgCtrlID(AFX_IDW_PANE_FIRST + 1);

    SetActiveView(m_pDefaultView);
    pDoc->RemoveView(m_pItalicsView);
    RecalcLayout();
}
```

You can choose to leave the inactive view in the application, or you can remove the inactive view from your application. The previous sample code uses **CDocument::RemoveView** to remove the inactive view. Removing it means that when the document object is changed and **CDocument::UpdateAllViews** is called, the inactive view object's **OnDraw** function will not be called.

Removing the inactive view is not required, but it makes the application more efficient. If the inactive view is not removed from the document object's list of attached views, nothing detrimental will happen when the inactive view's **OnDraw** function executes. In an application with only two views, each with simple **OnDraw** functions, you will not be able to tell much difference.

Notes

12. Enable and disable the two menu items, as shown in the following example code:

```
void CMainFrame::OnUpdateViewDefault(CCmdUI* pCmdUI)
{
    // Only enable the default view menu if the current view
    // is the italics view. The isKindOf function checks if the
    // active view is derived from CItalicsView
RUNTIME_CLASS macro
    pCmdUI->Enable(
        GetActiveView()->IsKindOf(RUNTIME_CLASS(CItalicsView)));
}

void CMainFrame::OnUpdateViewItalics(CCmdUI* pCmdUI)
{
    // Only enable the italics view menu if the current view
    // is the default view.
    pCmdUI->Enable(
        GetActiveView()->IsKindOf(RUNTIME_CLASS(CDefaultView)));
}
```

The code samples discussed in this topic appear in the sample application SDI2VIEWS, which is located in the folder \Samples\Ch04.

Notes

89 Adding Multiple Views to an MDI Application

In this section, you will learn how to create an MDI application with two or more views of the same document. The views are constructed from different classes, each view providing a different way to view the data stored in the document.

This section includes the following topics:

- Building an MDI Application
- Starting an Application with the Default View
- Selecting a View for Application Startup

Notes

90 Building an MDI Application

An MDI application provides the interface to open and work on more than one document simultaneously. You can create multiple views of different types and attach them to these documents.

▶ **To build an MDI application with two views**

1. Build an MDI application by using MFC AppWizard. AppWizard will create a skeleton MDI application with a single view.
2. Add a new class derived from the **CView** class.
3. Add debug and nondebug versions of the **GetDocument** function.

   Use the **GetDocument** functions from your existing view class as a model.
4. Include the header file for the document class in the source file of the new view class.
5. Place the command handlers in the **CChildFrame** class.

   It is easier to access the view or the document from **CChildFrame** rather than from the view class. Use the **CFrameWnd::GetActiveView** and **CFrameWnd::GetActiveDocument** functions to access the views and the document.
6. Add a handler for the **OnDraw** event of the new view class.

   At this point, you must decide whether you want users to select the type of view at application startup, or whether you want to have your application start with a default view. In either case, the user can select another view at run time.

Notes

91 Starting an Application with the Default View

If the users of your application are likely to use one view more often than another, you can define the frequently used view as the default view. When you provide a default view for your application, you must manage the document template without the help of ClassWizard.

▶ **To start an application with a default view**

1. Add a variable to the **CMainFrame** class in your application. For example:

```
class CMainFrame : public CMDIFrameWnd
{
    ...
protected:
    CDocTemplate* m_pItalicsTemplate;
    ...
}
```

2. Initialize the variable in the constructor of **CMainFrame** as shown in the following example:

```
CMainFrame::CMainFrame()
{
    m_pItalicsTemplate = 0;
}
```

3. Add a menu item for a visible child window.

   Add a menu item to be used when the application is displaying a visible child window. You can choose the menu to which you add the item; however, do not place the item in the IDR_MAINFRAME menu. The **Window** menu is a good choice, because that is where MFC AppWizard places the menu item that creates another window of the same type as the default.

Notes

4. Add a command handler for the new menu item to the **CMainFrame** class.

   To see sample code that shows the menu command handler, click this icon.
    (CD-ROM displays the following sample code.)

```
// Menu Command Handler for displaying Italics View
// Samples\CH04\MDI2Views
void CMainFrame::OnWindowItalics()
{
    if (0 == m_pItalicsTemplate)
    {
        m_pItalicsTemplate = new CMultiDocTemplate(
            IDR_XXXXTYPE,
            RUNTIME_CLASS(CXXXXViewsDoc),
            RUNTIME_CLASS(CChildFrame),
            RUNTIME_CLASS(CItalicsView));
    }

    CMDIChildWnd* pActiveChild = MDIGetActive();
    CDocument* pDocument;
    if (NULL == pActiveChild ||
      (pDocument = pActiveChild->GetActiveDocument()) == NULL)
    {
        TRACE0("Warning: No active document.\n");
        AfxMessageBox(AFX_IDP_COMMAND_FAILURE);
        return;     // command failed
    }

    CFrameWnd* pFrame = m_pItalicsTemplate->
                    CreateNewFrame(pDocument, pActiveChild);
    if (NULL == pFrame)
    {
        TRACE0("Warning: failed to create new frame.\n");
        return;     // command failed
    }

    m_pItalicsTemplate->InitialUpdateFrame(pFrame, pDocument);
}
```

   The code in the previous example determines when the template object needs to be created. It then obtains pointers to the current document, and the current active child window. These two pointers are needed to call the **CDocTemplate::CreateNewFrame** function. The **CDocTemplate::InitialUpdateFrame** function is called to finish creating the document template.

Notes

5. Destroy any dynamically allocated objects.

   In this process, some objects have been dynamically allocated and must be destroyed. The following example code shows the destructor in **CMainFrame**:

```
CMainFrame::~CMainFrame()
{
    if  (0 != m_pItalicsTemplate)
    {
        delete m_pItalicsTemplate;
        m_pItalicsTemplate = 0;
    }
}
```

The code samples discussed in this topic appear in the sample application MDI2VIEWS, which is located in the folder \Samples\CH04.

Notes

92 Selecting a View for Application Startup

Rather than define a default view for documents, you can provide users with an option to select the view at application startup. This will also provide the user with an option for a new view when selecting **New** on the **File** menu.

▶ **To select a view for application startup**

1. Add another document template to the **CWinApp** class. Store pointers to the new and existing templates in the application class as shown in the following example code:

```
class CMDI2ViewsBApp: public CWinApp
{
public:
    CMultiDocTemplate * m_pItalicsTemplate;
    CMultiDocTemplate * m_pDefaultTemplate;
    ...
}
```

2. Modify the **InitInstance** function in the application class. For example:

```
BOOL CMDI2ViewsBApp::InitInstance()
{
    ....
    m_pDefaultTemplate = new CMultiDocTemplate(
        IDR_MDI2VITYPE,
        RUNTIME_CLASS(CMDI2ViewsBDoc),
        RUNTIME_CLASS(CChildFrame),
        RUNTIME_CLASS(CDefaultView));
    AddDocTemplate(m_pDefaultTemplate);
    m_pItalicsTemplate = new CMultiDocTemplate(
        IDR_MDI2VITYPE2,
        RUNTIME_CLASS(CMDI2ViewsBDoc),
        RUNTIME_CLASS(CChildFrame),
        RUNTIME_CLASS(CItalicsView));
    AddDocTemplate(m_pItalicsTemplate);
    ....
}
```

   The application framework takes care of any cleanup for these dynamically allocated objects.

3. Create a menu, icon, and document template string for the new document template.

   The second template needs its own menu, icon, and document template string. Copy the existing IDR_XXXXTYPE menu and icon and give each an appropriate name, such as IDRXXXTYPE2. Make any changes to the menu before copying it. The table entry for the document template string must be changed slightly. If the string IDR_XXXXTYPE looks like this:

   ```
   \nXXXX\nDefault\n\n\nXXXXViews.Document\nXXXX Document
   ```

   Then the string IDRXXXTYPE2 must look like this:

   ```
   \nXXXX\nItalics\n\n\nXXXXViews.Document\nXXXX Document
   ```

Notes

4. Add a menu entry to the Windows menu that will display the new view.

   Use ClassWizard to install a command handler for the new menu entry in the **CMainFrame** class. Then use ClassWizard to install a command handler for the **OnWindowNew** ID, and give the handler the same name and location as the handler that displays the other view. In effect, you will have two entries in the message map pointing to the same function.

   To see sample code that shows how to determine which menu invoked the command handler, click this icon.

    (CD-ROM displays the following sample code.)

```
// Generic command handler to handle view display in an application
// that lets the user choose the view on application startup and using
// a menu item
// Samples\CH03\MDI2VIEWSB
void CMainFrame::OnWindowItalics()
{
    // We'll need a pointer to the active child...
    CMDIChildWnd* pChild = MDIGetActive();
    if (NULL == pChild)
    {
        TRACE0("Warning: No active child.\n");
        AfxMessageBox(AFX_IDP_COMMAND_FAILURE);
        return;     // command failed
    }

    // and to the active document...
    CDocument* pDoc = pChild->GetActiveDocument();
    if (NULL == pDoc)
    {
        TRACE0("Warning: No active document.\n");
        AfxMessageBox(AFX_IDP_COMMAND_FAILURE);
        return;     // command failed
    }

    // and to the application object.
    CMDI2ViewsBApp * pApp = (CMDI2ViewsBApp *)AfxGetApp();

    // Since both menu selections end up here (see message map)
    MSG const * pMsg = GetCurrentMessage();
    CMultiDocTemplate * pTemplate;

    // we need to know how we got here to figure out which template to
use...
    if (LOWORD(pMsg->wParam) == ID_WINDOW_ITALICS)
        pTemplate = pApp->m_pItalicsTemplate;
    else
        pTemplate = pApp->m_pDefaultTemplate;
```

(Code continues on following page.)

Notes

```
    // so that finally we can create a new frame...
    CFrameWnd* pFrame = pTemplate->CreateNewFrame(pDoc, pChild);
    if (NULL == pFrame)
    {
        TRACE0("Warning: failed to create new frame.\n");
        return;     // command failed
    }

    // which is then updated.
    pTemplate->InitialUpdateFrame(pFrame, pDoc);
}
```

The code samples discussed in this topic appear in the sample application MDI2VIEWSB, which is located in the folder \Samples\Ch04.

Notes

93 Suppressing Automatic Creation of a Child Window

When MFC AppWizard generates an MDI application, it automatically creates a new child window. You can suppress this feature with an addition to the **InitInstance** member of the application class, as shown in the following example code:

```
CCommandLineInfo cmdInfo;
ParseCommandLine(cmdInfo);

//Insert this statement in your application:
if (CCommandLineInfo::FileNew == cmdInfo.m_nShellCommand)
    cmdInfo.m_nShellCommand = CCommandLineInfo::FileNothing;

if (!ProcessShellCommand(cmdInfo))
    return FALSE;
```

If the user specifies a parameter on the command line, then you would not want to change `m_nShellCommand`. The example code shows how to determine whether a command-line parameter was specified and skip the change to `m_nShellCommand` if a parameter was specified.

Notes

94 Setting Titles for Child Frame Windows

You can define the title that should be displayed on child windows when the user opens a document or chooses to open a new window for a document that is already open.

Setting Titles for New Documents

The default title of a child window in an MDI application is a concatenation of the document title and a sequence number. The source of the document title is the second substring of the document template string.

Note The document template string is the string table entry with the name IDR_XXXXXXTYPE. Set the contents of each of the IDR_XXXXXXTYPE substrings by clicking the **Advanced** button in Step 4 of MFC AppWizard. You can modify these substrings later by editing the string table. For more information, search for "CDocTemplate::GetDocString" and "CDocTemplate::SetDefaultTitle" in MSDN Library Visual Studio 6.0, or read the Knowledge Base article Q129095.

 (CD-ROM displays the following tip.)

The Microsoft Knowledge Base contains detailed technical articles and samples that are created and maintained by Microsoft Product Support Services (PSS). This database is updated daily and contains technical information about Microsoft products, fix lists, documentation errors, and answers to commonly asked technical support questions.

To search for Knowledge Base articles on the Microsoft Technical Support Web site, go to the topic Knowledge Base Articles in the Library.

The document class provides functions to access the document title: **CDocument::GetTitle** and **CDocument::SetTitle**. Typically, an application calls **CDocument::SetTitle** from the **OnNewDocument** function.

Setting Titles for New Windows

When you click **New Window** on an MDI application's **Window** menu, no new document is created, so **OnNewDocument** is not invoked. The title of the new window is the document title for that document object, with ":2" appended after the title. The existing document window has ":1" appended to its title. The titles of new windows on that document will be appended sequentially starting with ".3." When you close any window on that document, the framework renumbers all the remaining windows.

Notes

If the default numbering is not appropriate for your application, you can create your own titles for new windows. The steps necessary to change the title of an MDI child window frame are as follows:

1. Override the **PreCreateWindow** function of the child frame class, as shown in this example:

```
BOOL CChildFrame::PreCreateWindow(CREATESTRUCT &cs)
{
    cs.style&=~(LONG)FWS_ADDTOTITLE;
    if (!CMDIChildWnd::PreCreateWindow(cs))
        return FALSE;
    return TRUE;
}
```

2. Override the **OnInitialUpdate** function of the view class, as shown in this example:

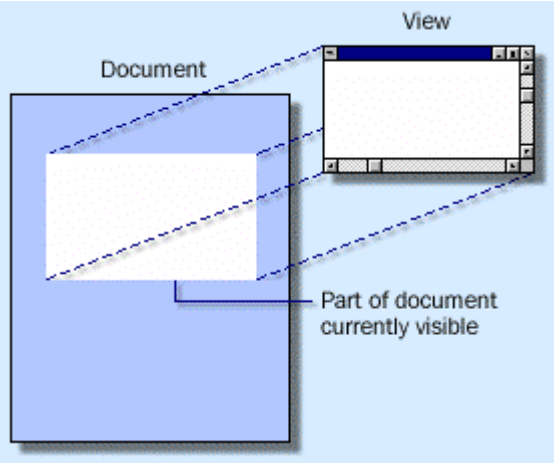
```
void CXXXXView::OnInitialUpdate()
{
    CView::OnInitialUpdate();
    GetParent()->SetWindowText(GetDocument()->GetTitle()+ " - This is a
test!");
}
```

Run the application. The first view displays "*XXXX*1 - This is a test!" Titles of other views will be identical except for the document name.

Notes

95 Adding Scrolling Views

A document has a logical size. A view, which is inherently limited to a physical area on a given device, can often display only part of the document. The following illustration shows a scrolling view and the underlying document.



The user controls how much of the document to view by shrinking or enlarging the viewport, and selects which part of the document to view by manipulating the horizontal and vertical scroll bars.

This section includes the following topics:

- Introduction to Scrolling Views
- Converting the Application View to a Scrolling View
- Adjusting Scrolling Characteristics
- Converting Coordinates

Notes

96 Introduction to Scrolling Views

MFC encapsulates the functionality of a scrolling view in the **CScrollView** class.

To see a demonstration of how to create an application with a scrolling view, click this icon.
 (CD-ROM plays the demonstration, "Creating a Scroll View Application.")

▶ **To add a scrolling view to your application**

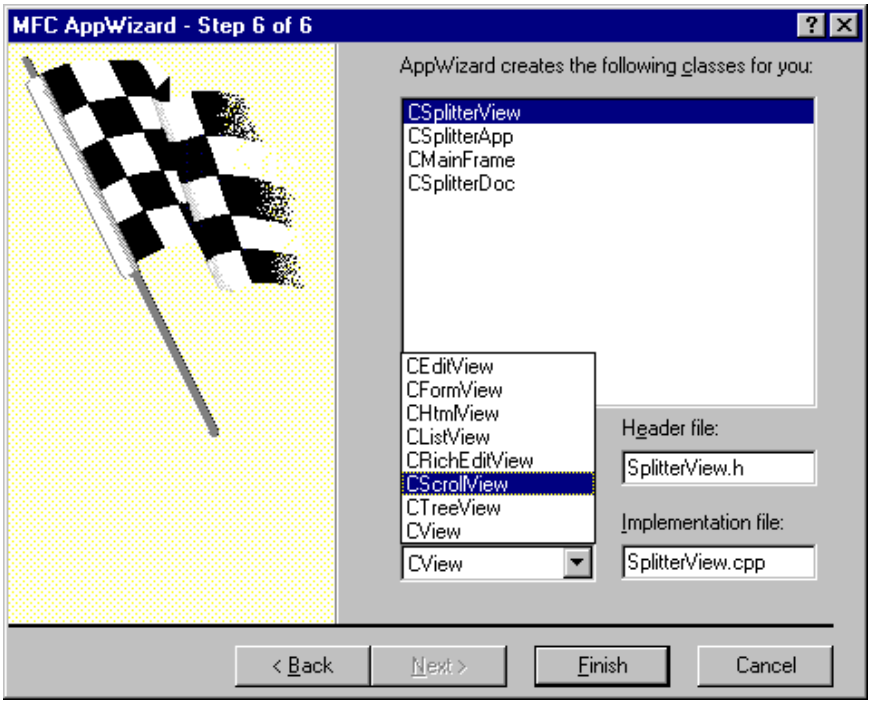
1. Set the application's base view class to **CScrollView**.
2. Adjust the scrolling characteristics to set the mapping mode and document size appropriately.
3. Handle the conversion between logical and physical (device) coordinates as required by your application.

Notes

97 Converting the Application View to a Scrolling View

You specify **CScrollView** as the base class for the view in your application to make it a scrolling view.

To see an illustration of the **MFC AppWizard - Step 6** dialog box with the base view class set to **CScrollView**, click this icon.
(CD-ROM displays the following illustration.)



If you have an existing application that uses **CView** and you want to convert it to a scrolling view, do the following:

- Replace all occurrences of **CView** with **CScrollView** in your existing view's header file.
- Replace all occurrences of **CView** with **CScrollView** in your existing view's implementation file.

Notes

98 Adjusting Scrolling Characteristics

You can adjust the scrolling characteristics of an application with the **OnInitialUpdate** or **OnUpdate** function of the application's view class. Call **CScrollView::SetScrollSizes** to set the mapping mode and the size of the document appropriately.

To see sample code that sets the mapping mode and document size in the **OnInitialUpdate** function, click this icon.

 (CD-ROM displays the following sample code.)

```
// Setting mapping mode and document size of an application with
// scrolling view
// Samples\CH04\Scroll

void CMyScrollView::OnInitialUpdate()
{
    CScrollView::OnInitialUpdate();
    CSize sizeTotal;

    sizeTotal.cx = sizeTotal.cy = 200;
    SetScrollSizes(MM_TEXT, sizeTotal);

    // Optional: To resize the frame to exactly fit the view at application
startup

    GetParentFrame()->RecalcLayout();
    ResizeParentToFit();
}
```

For information about scrolling functions, search for "Scrolling functions" in MSDN Library Visual Studio 6.0.

Notes

99 Converting Coordinates

When specifying view coordinates, you can generally use logical coordinates. However, there are times when your application receives information in physical coordinates. This often happens in an application with a scrolling view. When the user scrolls through the document, the viewport origin no longer coincides with the origin of the document—that is, the window origin in logical space.

For example, in mouse button event handlers, your application receives the position of the mouse in terms of physical coordinates stored in a **CPoint** object. You can use the **CDC** functions **DPtoLP** and **LPtoDP** to convert between the coordinate systems of logical and physical space as required by your application.

To see sample code that shows how to use the **DPtoLP** function in the **OnLButtonDown** handler to convert coordinates for an application using a scrolling view, click this icon.

 (CD-ROM displays the following sample code.)

```
// Converting physical coordinates to logical coordinates to implement
// a scroll view
// Samples\CH04\Scroll

void CMyScrollView::OnDraw(CDC* pDC)
{
    CScrollDoc * pDoc = GetDocument();
    CPoint pt = pDoc->GetLocation();
    pDC->TextOut(pt.x, pt.y, pDoc->GetPhrase());
}

// The CPoint argument represents the location of the mouse in
// device (physical) coordinates. OnDraw, which will be called
// as a result of the call to Invalidate at the end of this
// function, always draws to logical space. Therefore, the CPoint
// argument must be converted to logical coordinates before storing
// the new location into the document object. To show the necessity
// of doing this, comment out the first 3 lines of code in this
// function, re-compile and execute.
void CMyScrollView::OnLButtonDown(UINT nFlags, CPoint point)
{
    // Create a client dc that points to the current view
    CClientDC dc(this);

    // Given that this is a scroll view, call OnPrepareDC to
    // adjust attributes of the device context appropriately.
    OnPrepareDC(&dc);

    dc.DPtoLP(&point);
```

(Code continues on following page.)

Notes

```
    // Store the new location in the document and invalidate the view.
    CScrollDoc * pDoc = GetDocument();
    pDoc->SetLocation(point);

    Invalidate();

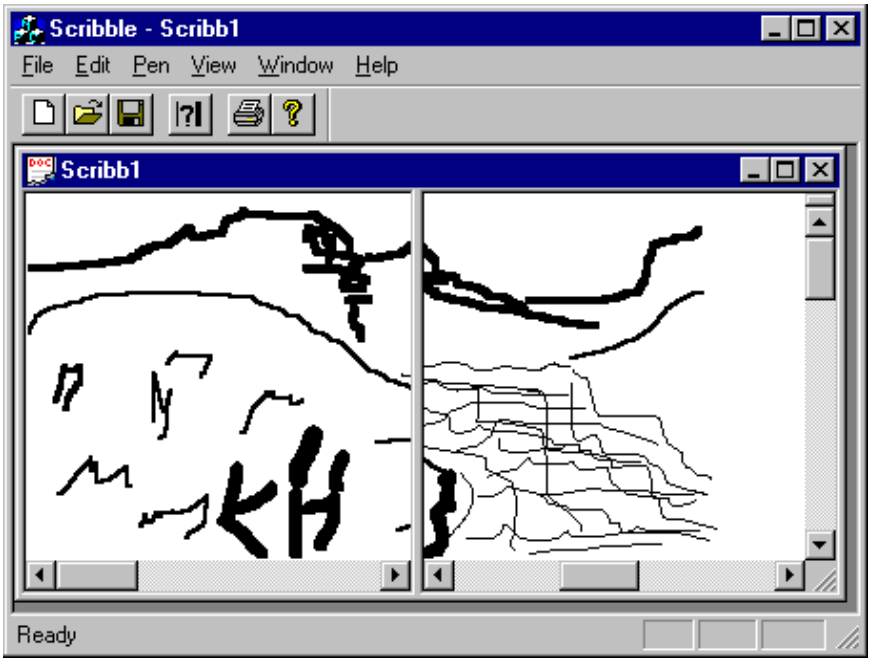
    CScrollView::OnLButtonDown(nFlags, point);
}
```

Notes

100 Implementing Splitter Windows

MFC provides the **CSplitterWnd** class for creating splitter windows.

To see an illustration of an application with splitter windows, click this icon.
 (CD-ROM displays the following illustration.)



This section begins with a discussion on the different types of splitter windows and proceeds to describe the steps in implementing them in your application.

This section includes the following topics:

- Introduction to Splitter Windows
- Creating Dynamic Splitter Windows
- Creating Static Splitter Windows

Notes

101 Introduction to Splitter Windows

There are two types of splitter windows: dynamic and static.

In dynamic splitter windows, additional panes are created and destroyed as the user splits and unsplits the views.

In static splitter windows, the panes are created when the window is created, and the order and number of panes never change. The panes are separated by a splitter bar that can change the relative sizes of the panes.

To see a demonstration that contrasts the features of the two types of splitter windows, click this icon.  (CD-ROM plays the demonstration, "Dynamic Vs. Static Splitter Windows.")

The following table describes the two types of splitter windows.

Characteristics	Dynamic splitter window	Static splitter window
Maximum number of panes	four (two rows x two columns)	256 panes (16 rows x 16 columns)
Different types of views	Can associate only a single type of view with all the panes.	Can associate different types of views with the panes.

Microsoft Excel, Word, and the source code editor provided with Microsoft Developer Studio all use dynamic splitter windows. Windows 95 Explorer is an example of an application that has a static splitter window.

For more information, search for "Splitter windows" in MSDN Library Visual Studio 6.0.

Notes

102 Creating Dynamic Splitter Windows

You can create a dynamic splitter window in the following ways:

- Use MFC AppWizard.
- Use the Components and Controls Gallery.
- Add code manually.

▶ To create a dynamic splitter window by using MFC AppWizard

- In MFC AppWizard, click **Advanced Options** in Step 4, click the **Window Styles** tab, select the **Use split window** check box, and click **Close**.

▶ To create a dynamic splitter window by using the Components and Controls Gallery

- In the Components and Controls Gallery, double-click the **Visual C++ Components** folder, click **Split Bars**, and click **Insert**.

▶ To create a dynamic splitter window by adding code manually

1. Add a **CSplitterWnd** object to the **CMainFrm** class in an SDI application (or to the **CChildFrame** class in an MDI application).
2. In the **OnCreateClient** function of the frame class, initialize the **CSplitterWnd** object by calling the **CSplitterWnd::Create** function.

When one of the views changes data in the document, use the **CDocument::UpdateAllViews** function to propagate the changes to the other panes.

To see a demonstration that shows how to create an application with a dynamic splitter window, click this icon.

 (CD-ROM plays the demonstration, "Creating a Dynamic Splitter Window.")

Notes

To see a sample code that shows how to implement a dynamic splitter window, click this icon.
 (CD-ROM displays the following sample code.)

```
// Implementing a dynamic splitter window in an MFC application
// Samples\CH04\Splitter
// MainFrm.h : interface of the CMainFrame class

class CMainFrame : public CFrameWnd
{
    ...
protected:
    CSplitterWnd m_wndSplitter;
    ...
};

// SAMPLES\CH03\splitter
// MainFrm.cpp : implementation of the CMainFrame class

BOOL CMainFrame::OnCreateClient(LPCREATESTRUCT lpcs, CCreateContext* pContext)
{
    CSize minWindow(10, 10);

    // These variables must both be 1 or 2 for dynamic splitters.
    int nRows = 2, nColumns = 1;

    return m_wndSplitter.Create(this, nRows, nColumns,
                                minWindow, pContext);
//
}
```

Notes

103 Creating Static Splitter Windows

A static splitter window is constructed along with the frame window at application startup. The user cannot create or destroy a static splitter window.

▶ **To create a static splitter window**

1. Add a **CSplitterWnd** object to the **CMainFrm** class in an SDI application (or to the **CChildFrame** class in an MDI application).

Note You can use MFC AppWizard or the Component and Controls Gallery to add the splitter object. However, you will need to replace the code generated by MFC AppWizard or the Component and Controls Gallery to make it appropriate for a static splitter window.

---

2. Add the classes for the additional views that you want in your application.

Note You can use ClassWizard to add new view classes.

---

3. Associate each view with the appropriate pane of the splitter window.

When you use MFC AppWizard to create an application, you choose the base class for the view in Step 6. MFC AppWizard adds this single view class to your application. You can then use ClassWizard to derive other view classes from appropriate view base classes and display each in a pane of a static splitter window.

▶ **To implement a static splitter window in an application**

1. Initialize the **CSplitterWnd** object by calling the **CSplitterWnd::CreateStatic** function in the **OnCreateClient** function of the frame class.
2. Associate the appropriate views with each pane. To do so, call the **CSplitterWnd::CreateView** function for each pane that you created in the call to **CSplitterWnd::CreateStatic**.

Notes

To see sample code that shows how to create and implement a static splitter window by using the **OnCreateClient** function, click this icon.

 (CD-ROM displays the following sample code.)

```
// Implementing an application with a static splitter window
// Samples\CH04\Splitter
// MainFrm.h : interface of the CMainFrame class

class CMainFrame : public CFrameWnd
{
    ...
protected:
    CSplitterWnd m_wndSplitter;
    ...
};

// Samples\CH03\Splitter
// MainFrm.cpp : implementation of the CMainFrame class

BOOL CMainFrame::OnCreateClient(LPCREATESTRUCT lpcs, CCreateContext* pContext)
{
// ...
    // This is the code for a static splitter.
    CRect cr;
    GetClientRect(&cr);

    CSize paneSize(cr.Width(), cr.Height() / 2);

    int rc;
    m_wndSplitter.CreateStatic(this, 2, 1);

    rc = m_wndSplitter.CreateView(0, 0,
                                  RUNTIME_CLASS(CSplitterView),
                                  paneSize, pContext);
    if (FALSE == rc)
        return rc;

    rc = m_wndSplitter.CreateView(1, 0,
                                  RUNTIME_CLASS(CItalicsView),
                                  paneSize, pContext);
    return rc;
}
```

Notes

Command Handler Issues

In a dynamic splitter application, command handlers are generally placed in the view class. The view class has the easiest access to the document, and commands often modify the data in the document.

However, a static splitter application generally has multiple views. Depending on what is appropriate for your application, you must decide where to place your command handlers. If you want the same commands enabled regardless of which view has focus, place the handlers in the frame class. If you want each view to have its own set of handlers, add the appropriate handlers to each view class.

Coordinating Updates in the Views

In applications that use different types of views, calling **UpdateAllViews** is not the easiest solution for updating data. For information about coordinating multiple views, see Coordinating Multiple Interrelated Views in this chapter.

Notes

105 Implementing Form Views

The **CFormView** class provides a window whose client area contains dialog box controls. These controls support entering, viewing, or altering data that is usually found in a form-based, data-access application.

In this section, you will look at creating and implementing form views derived from the **CFormView** class. This section also provides an overview of the database view classes.

This section includes the following topics:

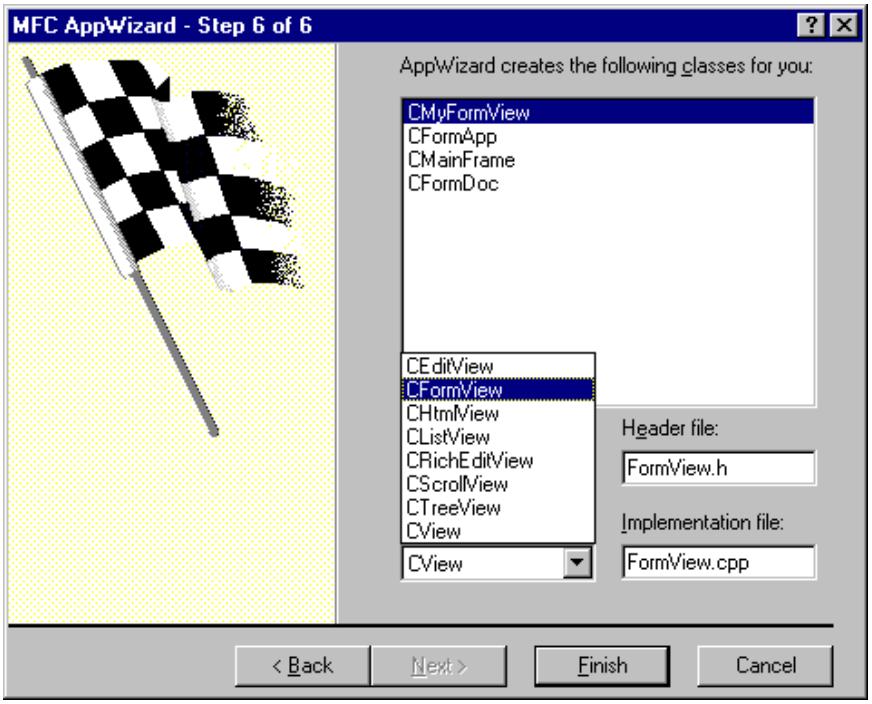
- User-Defined Form Views
- Database View Classes

Notes

106 User-Defined Form Views

To create a user-defined form view, you must create controls on the form, and handle updating the view and the document. Optionally, you can provide customized printing support.

Choosing **CFormView** as the base class for your application in Step 6 of MFC AppWizard provides a "blank" form, which you can customize with the appropriate controls. To see an illustration of the **MFC AppWizard - Step 6** dialog box with **CFormView** selected, click this icon.
 (CD-ROM displays the following illustration.)



Use the resource editor to open the dialog box resource in the .rc file that is associated with your application view, and to create the controls that you want on the form.

It is important to note that certain properties for the form itself are set by MFC AppWizard and should not be changed. The following table lists the required property settings for a form.

Property name	Value
Style	Child
Border	None
Titlebar	Off
Visible	Off

Notes

Use ClassWizard to add member variables to the form view class. For example, add **CString** variables for edit controls, **BOOL** variables for option buttons, and so on.

 (CD-ROM displays the following tip.)

If you have an existing Visual Basic-based application with a form similar to one that you want to use in your form view application, you can import the Visual Basic form into your Visual C++ project. For more information, search for "Using Custom Controls in a Dialog Box" and specifically, "Importing Visual Basic Forms." in MSDN Library Visual Studio 6.0.

Override the **OnInitialUpdate** function of your view class to perform the initialization you want to occur at application startup, such as resizing the frame to fit the view. Override the **OnUpdate** function of your view class to initialize the controls on the form with data from the document, as required.

Add message handlers in the view class to respond to messages generated by the controls contained in the view. Typically, you add code to the message handlers to move data from your view to your document. For example, if your document class contains a string, you would place an edit box on the form. Next, in the **OnUpdate** function, you would initialize the edit control with the current value of the string. Finally, in the handler for the EN_CHANGE message for each edit control, you would add the code to store the new string value in the appropriate member variable in the document.

Notes

To see sample code that implements a form view with an edit control and three check boxes, click this icon.

 (CD-ROM displays the following sample code.)

```
// Samples\CH04\Form
// FormView.cpp : implementation of the CMyFormView class

void CMyFormView::OnInitialUpdate()
{
    CFormView::OnInitialUpdate();

    // These next 2 calls make the main frame exactly the size
    // of form object.
    GetParentFrame()->RecalcLayout();
    ResizeParentToFit();
}

void CMyFormView::OnUpdate(CView* pSender, LPARAM lHint, CObject* pHint)
{
    // Obtain a pointer to the document object.
    // Note that OnDraw isn't used, since controls in a dialog
    // box can paint themselves.
    CFormDoc * pDoc = (CFormDoc *)GetDocument();

    // Set the dialog class's member variables to those in the document.
    m_dlgPhrase = pDoc->GetPhrase();
    m_blue = GetBValue(pDoc->GetColor()) == 255;
    m_green = GetGValue(pDoc->GetColor()) == 255;
    m_red = GetRValue(pDoc->GetColor()) == 255;

    // Cause DDX to occur from dialog object to dialog box.
    UpdateData(FALSE);
}
void CMyFormView::OnChangeWelcome()
{
    // Cause DDX to occur from dialog box to dialog object.
    UpdateData(TRUE);

    // Set the document's member variables to those in the dialog object.
    CFormDoc * pDoc = (CFormDoc *)GetDocument();
    pDoc->SetPhrase(m_dlgPhrase);
}
```

(Code continues on following page.)

Notes

```
void CMyFormView::OnColor(UINT nID)
{
    // Cause DDX to occur from dialog box to dialog object.
    UpdateData(TRUE);

    // Determine the state of the color check boxes, build an RGB
    // value, and store it back into the document.
    int r, g, b;
    r = m_red ? 255 : 0;
    g = m_green ? 255 : 0;
    b = m_blue ? 255 : 0;

    CFormDoc * pDoc = (CFormDoc *)GetDocument();
    pDoc->SetColor(RGB(r,g,b));

    // Now, invalidate the edit control so it repaints itself.
    CWnd * pWnd = GetDlgItem(IDC_WELCOME);
    pWnd->Invalidate();
}

HBRUSH CMyFormView::OnCtlColor(CDC* pDC, CWnd* pWnd, UINT nCtlColor)
{
    HBRUSH hbr = CFormView::OnCtlColor(pDC, pWnd, nCtlColor);

    // Note that this function is called before all controls on
    // the form are painted. The intent is to only change the
    // color of the edit box, so it's necessary to clarify for
    // which control this function is being called.
    if (GetDlgItem(IDC_WELCOME)->m_hWnd == pWnd->m_hWnd)
    {
        CFormDoc * pDoc = (CFormDoc *)GetDocument();
        pDC->SetTextColor(pDoc->GetColor());
    }

    return hbr;
}
```

Optionally, override the **OnPrint** function in your view class as required.

For more information, search for "CFormView" in MSDN Library Visual Studio 6.0.

Notes

107 Database View Classes

The **COleDBRecordView** class is derived from **CFormView** and displays database records in controls. For information about **COleDBRecordView**, see COleDBRecordView Class in Chapter 7: Using OLE DB Templates for Data Access.

In addition, MFC provides the following view classes:

- **CRecordView**

  Provides database forms for applications that use the MFC ODBC classes.
- **CDaoRecordView**

  Provides database forms for applications that use the MFC DAO classes.

Notes

108 Implementing Control Views

This section describes the view classes that are derived from the **CCtrlView** class and shows how to implement them in an application.

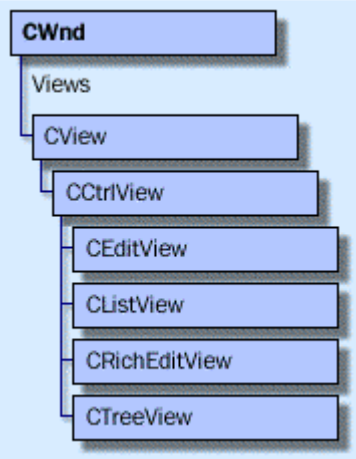
This section includes the following topics:

- Introduction to Control Views
- Overview of CListView
- Implementing CListView
- Overview of CTreeView
- Implementing CTreeView
- Overview of CEditView
- Implementing CEditView
- Overview of CRichEditView
- Implementing CRichEditView

Notes

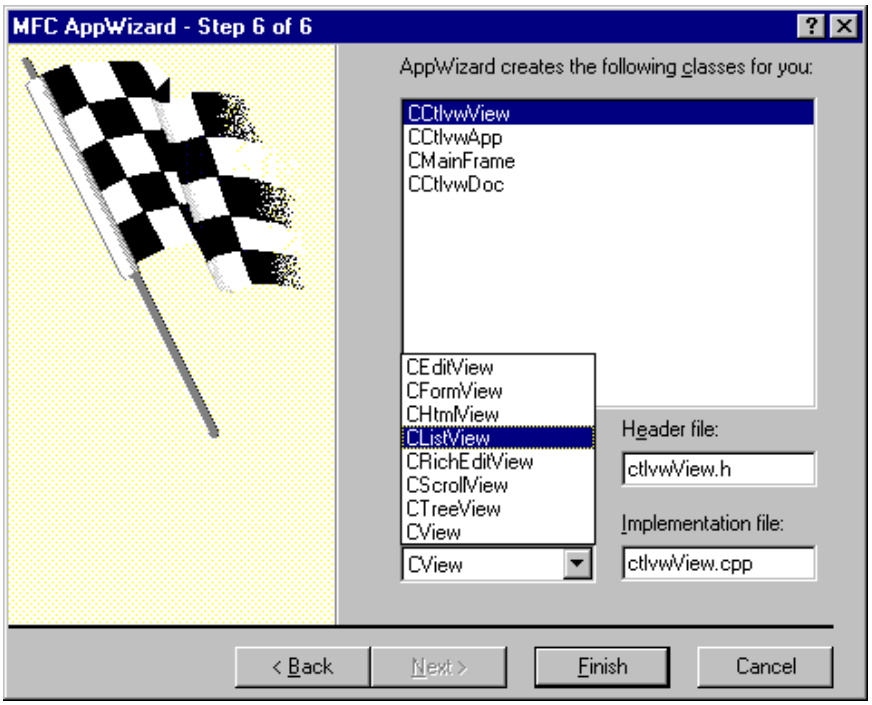
109 Introduction to Control Views

The **CCtrlView** class makes it possible to treat the client area of a view as if it were filled by a single control. The following illustration shows the classes derived from **CCtrlView**.



To see an illustration of the different types of views that you can select in the **MFC AppWizard - Step 6** dialog box, click this icon.

 (CD-ROM displays the following illustration.)



Notes

All four classes derived from **CCtrlView** have two member functions in common: a default constructor and an accessor function that returns a reference to the embedded common control object. With that reference, you can operate on the client area as if you were working with a single dialog box control.

The following example code uses the **GetListCtrl** accessor function in the **CListView** class to change the style of the list control that is embedded in the list view:

```
void CMyListView::OnInitialUpdate()
{
    // ...

    // Get a reference to the list control
    // embedded in the view
    CListCtrl & clc = GetListCtrl();

    // Use the reference to call member functions
    // of the embedded control
    clc.ModifyStyle(NULL, LVS_SORTASCENDING | LVS_REPORT);

    // ...do rest of initialization steps
}
```

When working with the **CCtrlView**-derived view classes, typically you will also use members of the control classes that they encapsulate. The available public members for these controls can be found in their respective classes, which are shown in the following table.

View class	Common control name	Common control class
CListView	List control	**CListCtrl**
CTreeView	Tree control	**CTreeCtrl**
CEditView	Edit control	**CEdit**
CRichEditView	Rich edit control	**CRichEditCtrl**

In many of the other view classes that are derived more directly from **CView**, typically you add your drawing code to the **OnDraw** member function of the view class.

The view classes with embedded controls handle most of the drawing for you in their customized versions of **OnDraw**. Typically, you do not override **OnDraw** to handle painting, but you may want to override it for printing/print preview. To handle updates by the user in a view, place code in the appropriate message handlers.

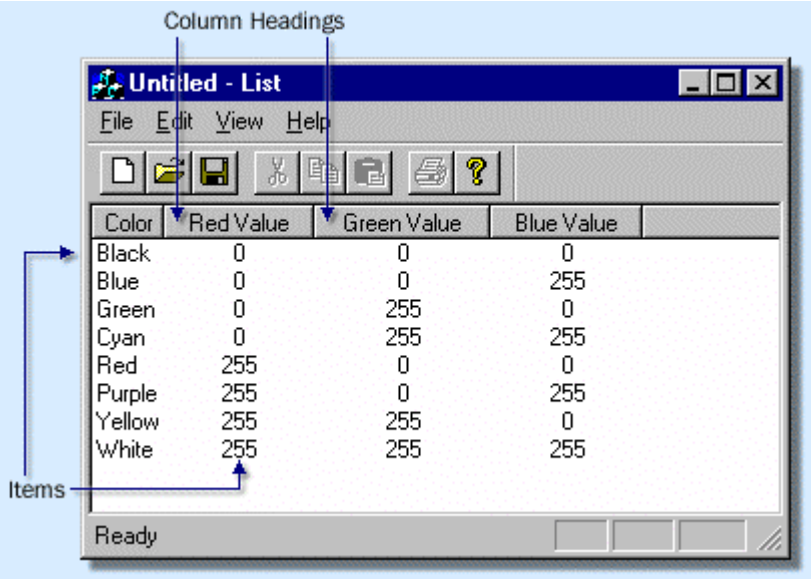
Notes

110 Overview of CListView

The **CListView** class simplifies the use of the list control and **CListCtrl**, the class that encapsulates list-control functionality, with the MFC document/view architecture.

A list control displays a collection of items, each consisting of an optional icon and a label. List views provide several ways of arranging items and displaying individual items. For example, additional information about each item can be displayed in columns to the right of the icon and label.

The following illustration shows an example of an application with a report-style list view.



The main steps in creating a list view application are as follows:

1. Run the MFC AppWizard. In Step 6, choose **CListView** as the base for your view class.
2. Edit the **OnInitialUpdate** function and add code to set the characteristics for the list view.
3. Edit the **OnUpdate** function and add code to populate the list.

For more information about **CListView**, search for "CListView" in MSDN Library Visual Studio 6.0.

Notes

111 Implementing CListView

To implement an application whose view class has been derived from **CListView**, you need to customize the characteristics of the list control and populate it.

Customizing a List Control

You customize a list control by editing the **OnInitialUpdate** function as follows:

1. Use the **GetListCtrl** function to get a reference to the list control that is embedded in the view, so that you can call the list control's **ModifyStyle** member function.
2. Use **ModifyStyle** to set the list view type. (The example in this section uses a report style.)
    (CD-ROM displays the following tip.)

   You can combine the LVS_SORTASCENDING or LVS_SORTDESCENDING value with other style values you pass as the second parameter to **ModifyStyles** to create a sorted list.

   ---

3. Add columns to the list control by creating a LV_COLUMN structure and setting the members with the values that your application requires. You can then set up a loop and use the **InsertColumn** member function of the list control object to insert each column with its specific characteristics, such as heading, format, width, and so on.

Notes

To see sample code that creates a four-column list in report style, click this icon.
 (CD-ROM displays the following sample code.)

```
// Setting the properties of a list view that displays a columnar
// report
// Samples\CH04\List
// ListView.cpp : implementation of the CMyListView class

void CMyListView::OnInitialUpdate()
{
    int i;
    CString s;
    LV_COLUMN lv;
    CListCtrl & clc = GetListCtrl();

    clc.ModifyStyle(NULL, LVS_SORTASCENDING | LVS_REPORT);

    lv.mask = LVCF_FMT | LVCF_TEXT | LVCF_WIDTH;
    lv.fmt = LVCFMT_LEFT;

    // Create 4 columns in the list control and
    // give each column a header string.
    for (i = ID_COLUMN1; i <= ID_COLUMN4; i++)
    {
        s.LoadString(i);
        lv.cx = 15 * clc.GetStringWidth(s) / 10;
        lv.pszText = (char *) (const char *)s;
        clc.InsertColumn(i - ID_COLUMN1, &lv);
        lv.fmt = LVCFMT_CENTER;
    }

    CListView::OnInitialUpdate();
}
```

Populating a List Control

You populate a list control by editing the **OnUpdate** function as follows:

1. Erase the outdated list.

   Use the **GetListCtrl** function to get a reference to the list control that is embedded in the view. Then call the list control's **DeleteAllItems** member function to prepare to refresh the list.

2. Populate the list with the current data, as follows:

   a. Create a new row and use the **InsertItem** member function of the list control to insert data into column 0.

   b. Use a loop to fill in the other columns of this row.

   c. Use the **SetItemText** member function to set the text of each column.

For more information about **CListView**, search for "CListView" in MSDN Library Visual Studio 6.0.

Notes

To see sample code that populates the rows in a list with information about RGB values for various colors, click this icon.

 (CD-ROM displays the following sample code.)

```
// Populating a list that displays a columnar report
// Samples\CH04\List
// ListView.cpp
void CMyListView::OnUpdate(CView* pSender, LPARAM lHint, CObject* pHint)
{
    CListCtrl & clc = GetListCtrl();
    CString s;

    // Remove all the list's data. Since OnUpdate will be called
    // when the list's data changes, this call is necessary before
    // rebuilding the list.
    clc.DeleteAllItems();

    // Each iteration of the loop builds 1 row. InsertItem fills
    // column 0, SetItemText fills the other columns.
    for (int i = ID_BLACK; i <= ID_WHITE; i++)
    {
        s.LoadString(i);
        // row will be 0-7, which corresponds to the 8 basic colors.
        int row = i-ID_BLACK;
        // insert the string
        clc.InsertItem(row, s);

        // Convert the color to its component RGB values.
        int mask = 4;
        for (int col = 1; col <= 3; col++)
        {
            // Determine if a bit is on or not.
            s.Format("%d", row & mask ? 255 : 0);
            clc.SetItemText(row, col, s);
            mask /= 2;
        }
    }
}
```

Notes

112 Overview of CTreeView

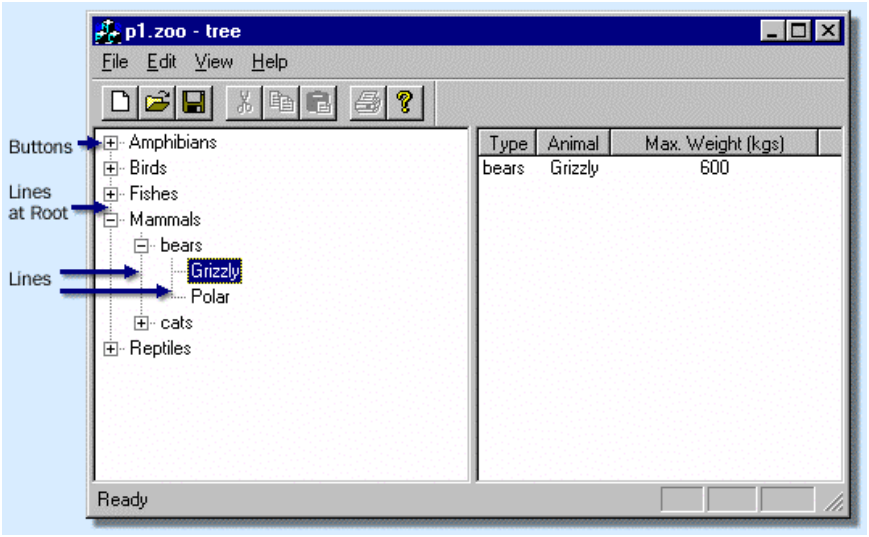
The **CTreeView** class simplifies the use of the tree control and **CTreeCtrl**, the class that encapsulates tree-control functionality.

A tree control is a window that displays a hierarchical list of items, such as the headings in a document, the entries in an index, or the files and folders on a disk. Each item consists of a label and an optional bitmapped image, and each item can have a list of subitems associated with it. By clicking an item, the user can expand and collapse the associated list of subitems.

Use **CTreeView** when you want the tree control to act like a view window in document/view architecture as well as a tree control. A tree view occupies the entire client area of a frame window or pane of a splitter window. It automatically resizes when its parent window is resized, and it can process command messages from menus, accelerator keys, and toolbars. Because a tree control contains the data necessary to display the tree, the corresponding document object does not have to be complex; you can even use **CDocument** as the document type in your document template.

The tree control (and therefore, the **CTreeCtrl** class) is available only to applications running under Windows 95 and Windows NT versions 3.51 and later.

To see an illustration of a tree view, click this icon.
 (CD-ROM displays the following illustration.)



To see a demonstration of how to create a tree view application, click this icon.
 (CD-ROM plays the demonstration, "Creating a Tree View Application.")

In the demonstration, the tree control has an image list associated with it. Associating an image list with the tree control is optional.

Notes

The main steps in creating a tree view application are as follows:

1. Run MFC AppWizard. In Step 6, choose **CTreeView** as the base for your view class.
2. Edit **OnInitialUpdate** and add code to set the characteristics for the tree view.
3. If you want to have images associated with the nodes, create an image list and associate it with the tree control.
4. Edit **OnUpdate** and add code to populate the tree.

For more information about **CTreeView**, search for "CTreeView" in MSDN Library Visual Studio 6.0.

Notes

113 Implementing CTreeView

To implement an application that uses a view based on **CTreeView**, you need to customize the properties of the tree control and populate it. If you want to display images in the tree view, then you must build an image list and associate it with the view.

Customizing Tree View Characteristics

Use **CTreeView::GetTreeCtrl** to call the **ModifyStyle** member function of the tree control that is embedded in the view. Use **ModifyStyle** to include buttons and horizontal and vertical lines in the tree control, as shown in the following example code:

```
void CTreeWithImagesView::OnInitialUpdate()
{
    ...
    GetTreeCtrl().ModifyStyle(NULL,
        TVS_HASBUTTONS | TVS_HASLINES | TVS_LINESATROOT);
    ...
}
```

Associating an Image List with the Tree Control

To display images with the nodes, you must complete the following steps:

1. Use the Resource editor to create the bitmaps to be used in the image list, and add them to your project.
2. Build an image list.
3. Associate the image list with the tree control.
4. As you insert a node, select an image to use for each of the two states of a node: selected and unselected.

Notes

To see sample code that builds an image list and associates it with the tree control, click this icon.
 (CD-ROM displays the following sample code.)

```
// Samples\CH04\Treewimg
// TimgView.cpp

void CTreeWithImagesView::InsertBitmaps()
{
    CImageList *cim;
    cim = new CImageList(); //dynamically allocate list
    cim->Create(BITMAP_WIDTH, BITMAP_HEIGHT, TRUE,
                NUM_BITMAPS, 0);

    CBitmap bitmap; // Temporary holding area for current
                    // for bitmap being loaded

    // Load the bitmaps and add them to the image list.
    for (int i = IDB_AMPHIBIAN; i <=
                    IDB_REPTILE_SELECTED; i++)
    {
        bitmap.LoadBitmap(i);
        cim->Add(&bitmap, (COLORREF)0xFFFFFF);
        bitmap.DeleteObject();
    }

    // Associate the image list with the tree control.
    GetTreeCtrl().SetImageList(cim, TVSIL_NORMAL);
}
```

Populating the Tree Control

To populate the tree control, add code to the **OnUpdate** function to refresh the control. Refreshing the tree control consists of two steps:

1. Delete all data from the tree control.
2. Populate the tree control (with the updated items).

Notes

To see sample code that populates the tree control, click this icon.
 (CD-ROM displays the following sample code.)

```
// Samples\CH04\Treewimg
// TimgView.cpp

void CTreeWithImagesView::InsertAnimals()
{
    static char * classes[] = {  "amphibians", "birds", "fishes",
                                 "mammals", "reptiles", NULL };
    static char * types[][4] = { "frogs", "toads", "salamanders", NULL,
                                 "eagles", "owls", "falcons", NULL,
                                 "trout", "perch", "bass", NULL,
                                 "bears", "whales", "rodents", NULL,
                                 "snakes", "turtles", "lizards", NULL };

    HTREEITEM hSubTree;
    int ImageListIndex = 0;

    CTreeCtrl & ctc = GetTreeCtrl();

    for (int r = 0; r < 5; r++)
    {
        hSubTree = ctc.InsertItem(classes[r], TVI_ROOT, TVI_SORT);
        ctc.SetItemImage(hSubTree, ImageListIndex, ImageListIndex + 1);
        // The string from the classes array is placed at the root...
        // and the newly-inserted node serves as parent to the types.
        InsertNodes(hSubTree, types[r], ImageListIndex, ImageListIndex + 1);
        ImageListIndex += 2;
    }

    static char * beartypes[] = { "grizzly", "polar", "black", NULL };
    HTREEITEM hSubTree2;
```

(Code continues on following page.)

Notes

```
    // Now that the tree is built, the bears node will have children
    // added to it. This requires 2 lookup operations.
    hSubTree = FindNode(ctc.GetRootItem(), classes[3]);
    if (NULL != hSubTree)
    {
        hSubTree2 = FindNode(hSubTree, types[3][0]);
        if (NULL != hSubTree2)
            InsertNodes(hSubTree2, beartypes, MAMMAL, MAMMAL + 1);
    }
}

// The function iterates over the children of ParentNode looking for a match.
// If ParentNode is NULL, the search starts in the root, otherwise it starts
// in the designated node. Returns NULL if not found, the HTREEITEM if found.
HTREEITEM CTreeWithImagesView::FindNode(const HTREEITEM ParentNode,
                                        const CString &str) const
{
    CTreeCtrl & ctc = GetTreeCtrl();
    HTREEITEM node;
    CString s;

    if (NULL == ParentNode)
        node = ctc.GetRootItem();
    else
        if (ctc.GetRootItem() == ParentNode)
            node = ParentNode;
        else
            node = ctc.GetChildItem(ParentNode);

    while (node != NULL)
    {
        s = ctc.GetItemText(node);
        // Halt the search when we find what we're looking for.
        if (0 == s.CompareNoCase(str))
            return node;
        node = ctc.GetNextItem(node, TVGN_NEXT);
    }
    // If we get to here, string was never found, and node == NULL.
    return node;
}
```

(Code continues on following page.)

Notes

```
// This function inserts the array of strings pointed to by the 2nd
// argument into the tree at the node represented by the first
// argument. All nodes have the same 2 image IDs.
void CTreeWithImagesView::InsertNodes(const HTREEITEM hSubTree, char **
AnimalClass,
                            const int ImageID, const int SelectedImageID)
{
    HTREEITEM node;
    CTreeCtrl & ctc = GetTreeCtrl();

    int i = 0;
    while (AnimalClass[i])
    {
        // Insert a node as a child of hSubTree and set its images.
        node = ctc.InsertItem(AnimalClass[i++], hSubTree, TVI_SORT);
        ctc.SetItemImage(node, ImageID, SelectedImageID);
    }
}
```

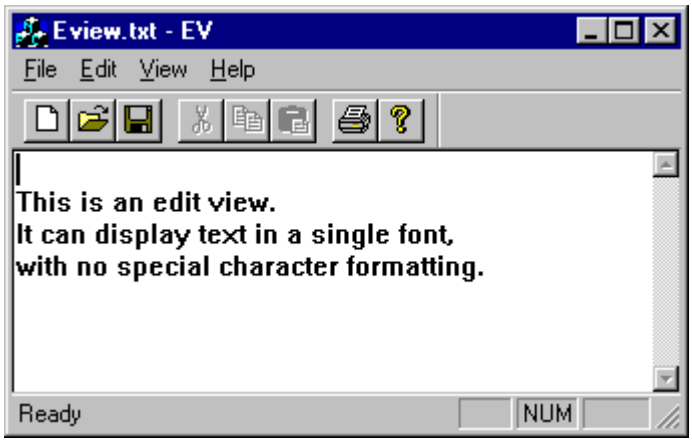
This sample code does not enable the user to change the data that is associated with the control.

Notes

114 Overview of CEditView

The **CEditView** class provides the functionality of a **CEdit** control with the following enhanced editing features: printing; find and replace; **Cut**, **Copy**, **Paste**, **Clear**, and **Undo** commands; and **File Save** and **File Open** commands. You can use **CEditView** to implement a simple text-editor view.

The following illustration shows an edit view application.



Notes

115 Implementing CEditView

Objects of type **CEditView** (or of types derived from **CEditView**) have the following limitations:

- **CEditView** does not implement true WYSIWYG (what you see is what you get) editing.

  Where there is a choice between readability on the screen and matching printed output, **CEditView** opts for screen readability.
- **CEditView** can use only one font to display text.

  No special character formatting is supported. Use the **CRichEditView** class for greater capabilities.
- The amount of text that a **CEditView** object can contain is limited. It can store only 64KB of text.


(CD-ROM displays the following tip.)

Note that you can make the edit control "read-only" by calling the **SetReadOnly** member function of your application's **CEdit** class in the view's **OnInitialUpdate** function.

Note If you want support for serialization as well as the behavior of the **CEdit** control, use a **CEditView**-derived view instead of an edit control on a dialog box or form view.

For more information about **CEditView**, search for "CEditView" in MSDN Library Visual Studio 6.0.

Notes

116 Overview of CRichEditView

A rich text file is a file that contains encoded, formatted text and graphics for easy transfer between applications. The rich-text encoding format is commonly used by document-processing programs such as Microsoft Word for Windows and for generating online Help files. Rich-text format files usually have an .rtf file extension.

A rich edit control is a window in which the user can:

- Enter and edit text that can be assigned character and paragraph formatting.
- Include embedded ActiveX objects.

The rich edit control covers a large subset of the full RTF specification.

For more information about **CRichEditView**, search for "CRichEditView" in MSDN Library Visual Studio 6.0.

Notes

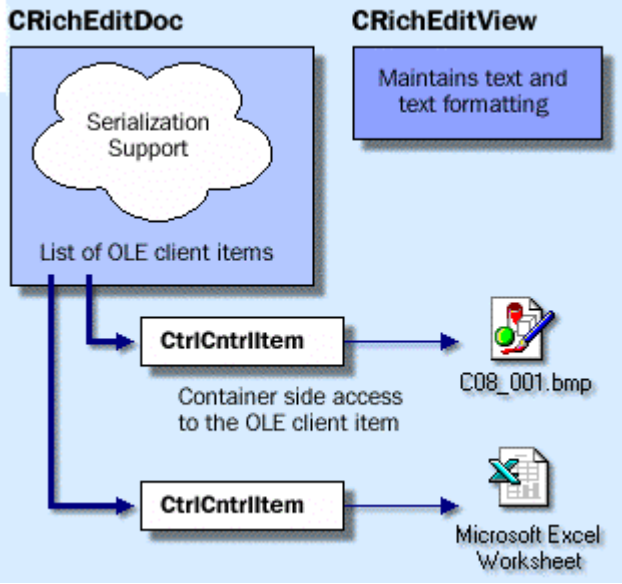
117 Implementing CRichEditView

The MFC class **CRichEditCtrl** encapsulates the rich edit control. The rich edit control (and therefore the **CRichEditCtrl** class) is available only to applications running under Windows 95 and Windows NT versions 3.51 and later.

The **CRichEditView**, **CRichEditDoc**, and **CRichEditCntrItem** classes provide the functionality of the rich edit control within the context of MFC's document/view architecture as follows:

- **CRichEditView** maintains the text and formatting characteristic of text.
- **CRichEditDoc** maintains the list of ActiveX client items that are in the view and, like any **CDocument**-derived object, provides support for serialization.
- **CRichEditCntrItem** provides container-side access to the ActiveX client item.

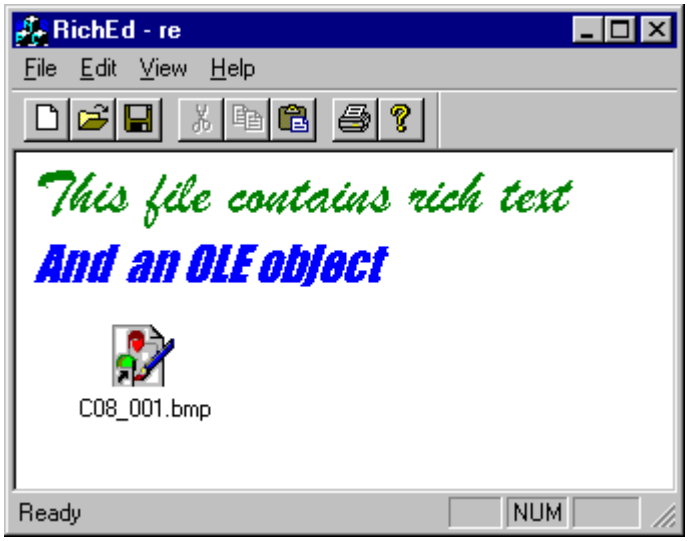
The following illustration shows how these classes interrelate.



The **CRichEditView** class provides numerous member functions that are wrappers for member functions of the **CRichEditCtrl** class. You use these wrappers to simplify the code that implements a given functionality.

Notes

If you use MFC AppWizard to generate an application that uses **CRichEditView** as the base class for the view, MFC AppWizard will change the base class for the document and will also add the class required for ActiveX container support. The following illustration shows an example of a rich edit view application.



Rich edit controls provide a programming interface for formatting text. However, you must implement any user-interface components necessary to make formatting operations available to the user.

Implementing the Drag-and-Drop Feature

The drag-and-drop technique is used to transfer data between applications, between windows in an application, or within a window. An MFC AppWizard-generated application that uses **CRichEditView** as the base class for its view includes drop target functionality. This means it can accept an OLE object that is dropped into a view.

For example, the user might select several files in the Windows 95 Explorer and drop them into the view of your application. You must add one or more handlers to your application to provide the appropriate response.

If your application contains a single view, add an **OnDropFiles** handler for the WM_DROPFILES message to your view class. If your application contains multiple views, consider having each view's **OnDropFiles** function send a WM_DROPFILES message to the frame class and implement the common functionality for handling dropped files in the frame's **OnDropFiles** function.

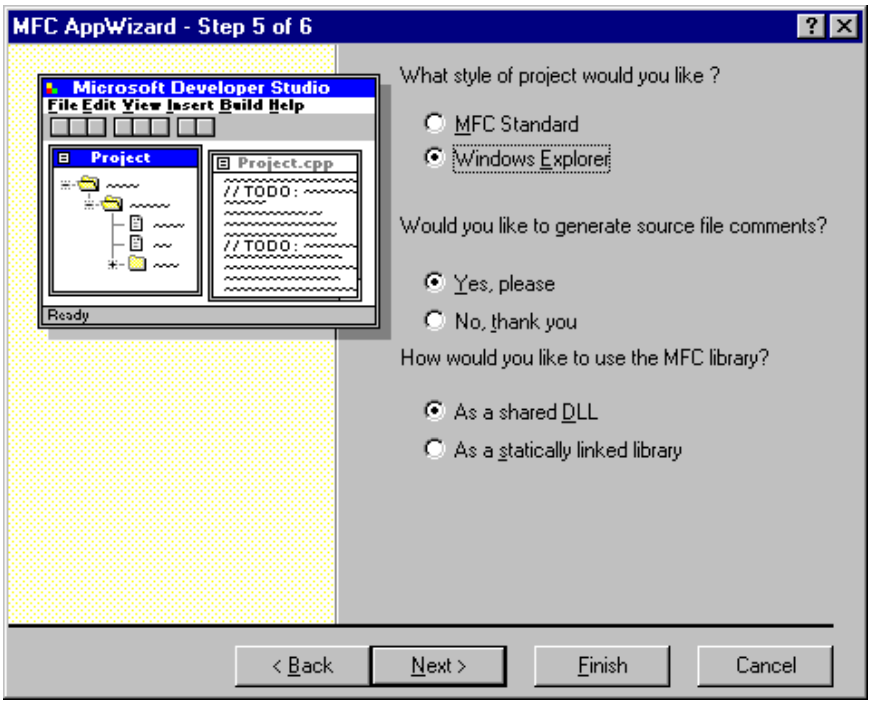
Notes

118 Creating an Explorer-Style Application

Many Windows system applications use the Explorer user interface, in which the client area is split vertically. The left side of the client area shows an overview pane, which makes navigation and browsing possible, while the right side of the client area shows details about the selection in the left pane.

You can create an Explorer-Style application by using MFC AppWizard and selecting the **Windows Explorer** option in Step 5.

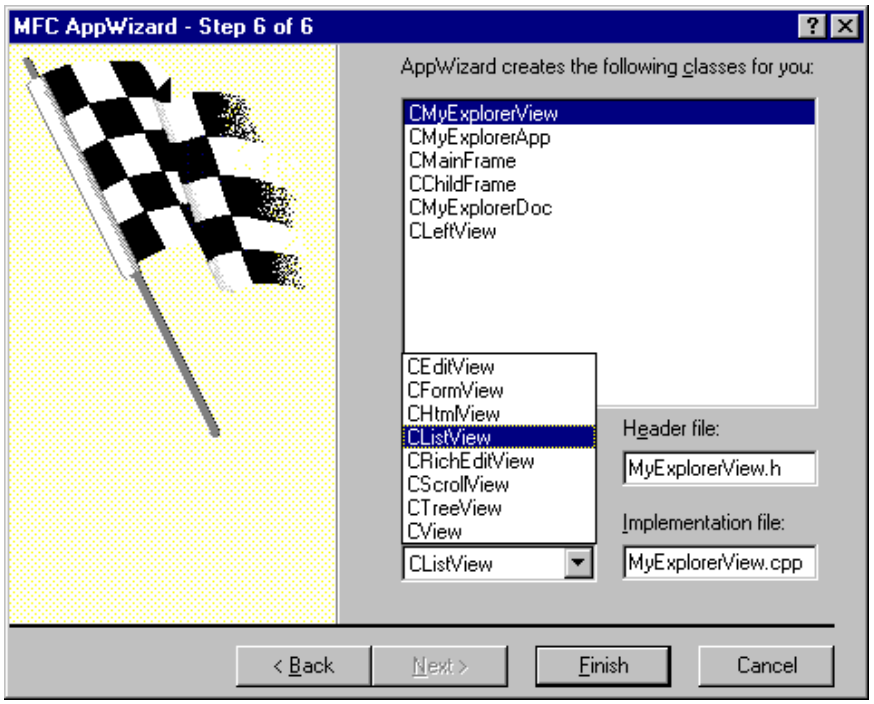
To see an illustration of the **MFC AppWizard - Step 5** dialog box, click this icon.
 (CD-ROM displays the following illustration.)



MFC AppWizard generates code so that each frame window that the application creates is split vertically using the **CSplitterWnd** object. The left pane always contains a tree view control. The right pane contains a list view control by default. You can modify the view type of the right pane to customize the display detail of the selected item in the left pane.

Notes

To see an illustration of the **MFC AppWizard - Step 6** dialog box with a list of view types, click this icon. (CD-ROM displays the following illustration.)



If you use the MFC AppWizard default list view in the right pane, the wizard creates additional menu choices and toolbar buttons to switch the view's style among report mode, small icon mode, and list mode.

Notes

119 Coordinating Multiple Interrelated Views

If your application uses two or more views of different types, you need to ensure that the display of data in the views is always synchronized.

There are two methods for coordinating the data in the views. The first method uses the **UpdateAllViews** function to update the views, whereas the second method updates the views without using this function. Which method you use depends on the requirements of your application.

Coordinating Data Using the UpdateAllViews Function

Using **UpdateAllViews** takes advantage of the document/view architecture as provided by MFC. Suppose that the user does something in a list view that causes a change in a form view. The list view packages the information it needs to pass to the form in a class derived from **CObject**. The list then calls **UpdateAllViews**, passing three parameters: a pointer to itself, NULL (0), and a pointer to the object where information is packaged.

The document object then calls the form's **OnUpdate** function, and passes the second and third arguments of **UpdateAllViews** to the form. The document does not call the list's **OnUpdate** function, because the list specified a pointer to itself in the first argument to **UpdateAllViews**.

Coordinating Data Without Using the UpdateAllViews Function

The second method requires you to manually control the process of coordinating the data. With this method, the document stores pointers to the two view objects. Each view initializes this pointer to itself in its **OnCreate** function. Each view also has as many public functions, with as many parameters as are needed, to respond to changes requested by the other view. When, for instance, the list needs to cause the form to change its display, the list obtains the pointer to the form from the document and calls a specific function in the form, passing arguments as required. Using this method may be much easier than packaging information in a class and passing that class by using the **UpdateAllViews** function as described in the first method.

Notes

121 Sample Applications

The following table describes the sample applications related to this chapter. These sample applications are located in the folder \Samples\Ch04.

Sample application subfolder	Description of application
Mdiapp	Shows how document class tracks all views attached to that class.
SDI2VIEWS	An SDI application with two views available. User can toggle between those two views at run time.
MDI2VIEWS	Shows how to implement an MDI application with two views of its data. Defaults at startup to one view, and the user can choose another from a menu selection.
MDI2VIEWSB	Similar to MDI2VIEWS, except that the user is asked when the application starts, and when opening a new document, which view to use.
List	Implements an SDI application with its view based on **CListView**. Shows how to populate a report-style list control.
Form	Implemented using **CFormView** as the base class for the view. Also shows how to change text color in an edit box.
Scroll	An SDI application with **CScrollView** as its base class. Shows how to adjust the frame to fit the view. Also shows how converting device points to logical points.
Splitter	An SDI application with a static splitter bar and different views in each pane. **CMainFrame**::**OnCreateClient** also contains commented source code for a dynamic splitter.
Tmpgraph	An SDI application with a static splitter bar and different views in each pane. Shows the document/view concept of different views of data. Uses persistence. Demonstrates minimal graphing capabilities.
Treewimg	An SDI application that uses **CTreeView** as the base for its view class. Shows how to create a **CImageList** and attach it to a tree control. Also shows how to populate a tree control.
Treelist	An SDI application that shows how to build an application with an interface like Windows Explorer: a vertical splitter and a tree view in its left pane and list view in its right pane.

Notes

124 Self-Check Questions

1. What is the relationship between views and other framework classes?

- A. A view is owned by the mainframe object.
- B. A view is embedded in its corresponding document object.
- C. A view is refreshed by the mainframe object.
- D. A view displays the contents of a document object.

2. How do you enhance an existing application to support a scrolling view?

- A. By adding code to the **OnInitialUpdate** function
- B. By setting the base class of the view to **CScrollView**
- C. By calling **CView::SetScrollSizes**
- D. By adding a scrollbar object to the view

3. What differentiates a dynamic splitter window from a static splitter window?

- A. Static splitters cannot have different views associated with each pane, whereas dynamic splitters can.
- B. Dynamic splitters have no maximum number of panes, but static splitters are limited to 256 panes.
- C. You must declare a **CDynSplitterWnd** object for a dynamic splitter and a **CSplitterWnd** object for static splitters.
- D. Dynamic splitter windows can be destroyed at run time, whereas static splitter windows cannot.

4. Which property name-value pairs is set by AppWizard for a form view?

- A. Style: Popup
- B. Border: None
- C. Titlebar: On
- D. System Modal: On

5. Which class is used as the base class for control views?

- A. **CControlView**
- B. **CCtrlView**
- C. **CListView**
- D. **CView**

Notes